Universidade do Minho

Ano Letivo: 2026/27

Sistemas Conexionistas para Aprendizagem Automática

PL2 – Data Exploration and Preparation



Licenciatura em Ciência de Dados, 1º Semestre, 3º Ano

Sumário

1 Data Understanding

2 Data Preparation

3 Univariate Analysis

4 Multivariate Analysis

Ficha de Exercícios 1

Flights Dataset

Label Encoding em Python

- Exemplo prático com *dataset* de voos (*flights_dataset.csv*)
- Objetivo: transformar *features* categóricas para usar em modelos de ML

Importações e leitura

```
import numpy as np
import pandas as pd

df = pd.read_csv('flights_dataset.csv')
print(df.info())
```

```
RangeIndex: 24604 entries, 0 to 24603
Data columns (total 16 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   year       24604 non-null  int64
 1   month      24604 non-null  int64
 2   day        24604 non-null  int64
 3   dep_time   24359 non-null  float64
 4   dep_delay  24359 non-null  float64
 5   arr_time   24323 non-null  float64
 6   arr_delay  24287 non-null  float64
 7   carrier    24604 non-null  object
 8   tailnum    24523 non-null  object
 9   flight     24604 non-null  int64
 10  origin     24604 non-null  object
 11  dest       24604 non-null  object
 12  air_time   24287 non-null  float64
 13  distance   24604 non-null  int64
 14  hour       24359 non-null  float64
 15  minute     24359 non-null  float64
dtypes: float64(7), int64(5), object(4)
```

Remover colunas não necessárias

```
df.drop(['hour','minute','tailnum'], axis=1, inplace=True)
```

— altera df sem criar cópia

Estas colunas são redundantes → *dep_time* contem a informação completa (hora e minuto).

tailnum é o identificador único do avião

Ficha de Exercícios 1

Flights Dataset

Inferir tipos e tratar valores em falta

```
df.infer_objects()

print(df.isnull().sum())
df.fillna(-99, inplace=True)
```

```
year          0
month         0
day           0
dep_time    245
dep_delay   245
arr_time    281
arr_delay   317
carrier       0
tailnum      81
flight        0
origin        0
dest          0
air_time    317
distance      0
hour        245
minute      245
```

Porque é que se substitui valores nulos (NaN) por -99?

→ É uma técnica de *masking*, para marcar valores em falta sem removê-los.

Atenção: pode distorcer modelos se não for bem interpretado (muitos algoritmos tratam -99 como valor real).

Que alternativas existem para lidar com valores em falta?

→ Remover linhas/colunas com muitos NaN.

→ Substituir pela média/mediana/moda.

→ Usar métodos de imputação mais avançados (KNN, regressão, etc.).

Ficha de Exercícios 1

➡ Flights Dataset

Explorar categorias (ex.: dest)

```
print(df['dest'].unique())
print('Unique count: %d' % df['dest'].value_counts().count())
print(df['dest'].value_counts())
```

- `unique()` mostra os valores distintos

```
['ANC' 'CLT' 'IAH' 'DTW' 'ORD' 'DEN' 'EWR' 'PHX' 'SLC' 'DFW' 'SJC' 'LAX'
 'OAK' 'SFO' 'SNA' 'SAN' 'ATL' 'BUR' 'MDW' 'IAD' 'PSP' 'MSP' 'HNL' 'JFK'
 'LAS' 'LGB' 'ONT' 'KOA' 'MKE' 'KTN' 'DCA' 'ABQ' 'MCO' 'OGG' 'MCI' 'PHL'
 'LIH' 'FLL' 'BOS' 'SMF' 'OMA' 'TUS' 'SBA' 'JNU' 'AUS' 'FAT' 'STL' 'RDM'
 'EUG' 'SAT' 'SEA' 'GEG' 'RNO' 'FAI' 'HDN' 'PDX' 'COS' 'MIA' 'LMT' 'CLE'
 'CVG' 'JAC' 'BWI' 'TPA' 'MSY' 'BNA' 'BLI' 'BOI' 'HOU' 'SIT']
```

- `value_counts()` mostra a frequência por categoria

```
dest
SFO    2012
LAX    1698
DEN    1472
PHX    1345
LAS    1215
       ...
HOU      10
BOI       9
CLE       9
JAC       5
SIT       4
```



Esta análise ajuda a decidir se as colunas têm alta cardinalidade (muitos valores) — importante para escolher entre *Label Encoding* e *One-Hot*.

Ficha de Exercícios 1

Flights Dataset

Estratégia do Label Encoding

```
def label_encoding(df):
    columns = df.columns.values
    df_labelled = df.copy()
    dictionary_of_dictionaries = {}
```

- columns: obtém o **nome** de todas as **colunas** do DataFrame.
- df_labelled = df.copy(): faz uma **cópia**, para não alterar o DataFrame original.
- dictionary_of_dictionaries = {}: cria uma estrutura para **guardar** os **mapeamentos** de todas as colunas, semelhante a:

```
{
  'carrier': {0: 'AS', 1: 'UA', 2: 'DL', ...},
  'origin': {0: 'SEA', 1: 'PDX'}
}
```

Função interna que converte cada valor de uma coluna categórica para o número correspondente no dicionário da coluna atual.

```
def convert_to_label(val):
    return column_dictionary[val]
```

Ficha de Exercícios 1

Flights Dataset

Estratégia do Label Encoding

```
for column in columns:
    column_dictionary = {}
```

1. Iterar por todas as colunas do DataFrame
2. Para **cada coluna**, cria um dicionário vazio (column_dictionary) onde irá armazenar os pares **categoria → número**.

```
if df[column].dtype != np.int64 and df[column].dtype != np.float64:
    unique_elements = df[column].unique()
    x = 0
    for unique in unique_elements:
        if unique not in column_dictionary:
            column_dictionary[unique] = x
            x+=1
```

1. Se a coluna **não** for numérica, obter **elementos únicos**
2. O loop cria um mapeamento:

- Primeiro valor encontrado → 0
- Segundo valor encontrado → 1
- E assim sucessivamente.

Ficha de Exercícios 1

Flights Dataset

Estratégia do Label Encoding

```
df_labelled[column] = list(map(convert_to_label, df[column]))
```

5. Substitui a coluna **original** pelos **números** mapeados.
6. map(convert_to_label, df[column]) aplica a função auxiliar para converter cada valor para o inteiro correspondente.

```
dictionary_of_dictionaries[column] = dict((v,k) for k,v in column_dictionary.items())
```

5.Guarda o **mapeamento inverso**: número → categoria. Isto permite decodificar no futuro.

```
return df_labelled, dictionary_of_dictionaries
```

5.No final, devolve:
df_labelled: DataFrame transformado (categorias → inteiros)
dictionary_of_dictionaries: todos os dicionários inversos (para recuperar os valores originais depois)

Ficha de Exercícios 1

Flights Dataset

Estratégia do Label Decoding

```
def label_decoding(df_labelled, label_dictionary):
    df_decoded = df_labelled.copy()
```

1. Recebe como entrada:
- df_labelled: DataFrame já transformado em inteiros.
- label_dictionary: o "dicionário de dicionários" que guarda os **mapeamentos inversos** (inteiro → categoria).
2. Cria uma cópia (df_decoded) para não alterar o DataFrame codificado.

```
    col = ''
    def decode_label(val):
        return label_dictionary[col][val]
```

3. Define uma **função auxiliar** (decode_label) que recebe um valor inteiro e o traduz de volta usando o dicionário inverso da coluna atual.
4. O truque aqui é a variável **col**: ela é atualizada em cada iteração do loop seguinte.

```
    for d in label_dictionary:
        col = d
        df_decoded[d] = list(map(decode_label, df_labelled[d]))
```

5. Percorre todas as **colunas** que foram **codificadas** (as chaves de label_dictionary).
6. **Atualiza** col para a coluna atual (d).
7. Usa **map(decode_label, df_labelled[d])** para **substituir** os valores **inteiros** da coluna pelos valores **originais** guardados em label_dictionary.
8. No final devolve um novo DataFrame (df_decoded) com as categorias restauradas.

Ficha de Exercícios 1

Flights Dataset

Como garantir que o mapeamento do *label encoding* é consistente se treinarmos e depois aplicarmos a novos dados?
→ Necessidade de guardar o dicionário de *encoding*, para que os mesmos valores sejam sempre convertidos para os mesmos números.

O que fazer se aparecer uma nova categoria não prevista?
→ Atribuir um valor especial (ex.: -1) ou atualizar o dicionário.

Porque é que precisamos do *decoding*?
→ Para interpretar os dados novamente na forma original.
Importante em relatórios ou quando o modelo precisa ser explicado.

Ficha de Exercícios 1

Flights Dataset

One-Hot Encoding com pandas

```
print('Unique Origin:', df['origin'].unique())

print(df.columns.values)

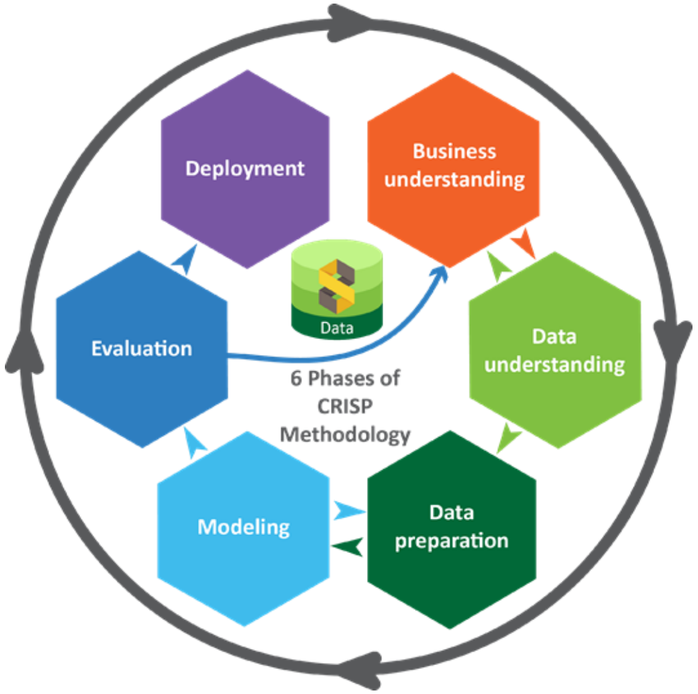
df_pandas_ohe = pd.get_dummies(df.copy(), columns = ['origin'], prefix = ['origin'])

print(df_pandas_ohe.head())
```

get_dummies cria colunas binárias para cada categoria de origin (ex.: origin_JFK, origin_LAX)

Data Understanding

O que é o *Data Understanding* dentro de um processo de Data Science?



⚠ É preciso olhar para os dados e perceber a sua estrutura, qualidade e integridade!

Data understanding é uma etapa grande do processo e, como tal, vai demorar! No entanto, dar-nos-á uma ideia do conjunto de dados, de cada variável, do seu significado e da sua relevância para o problema que estamos a trabalhar.

Objetivo: conhecer os dados disponíveis:

- entender o tipo de dados (numéricos, categóricos, temporais);
- quantidade de registos;
- quantidade de variáveis;
- formatos;
- granularidade;
- ...

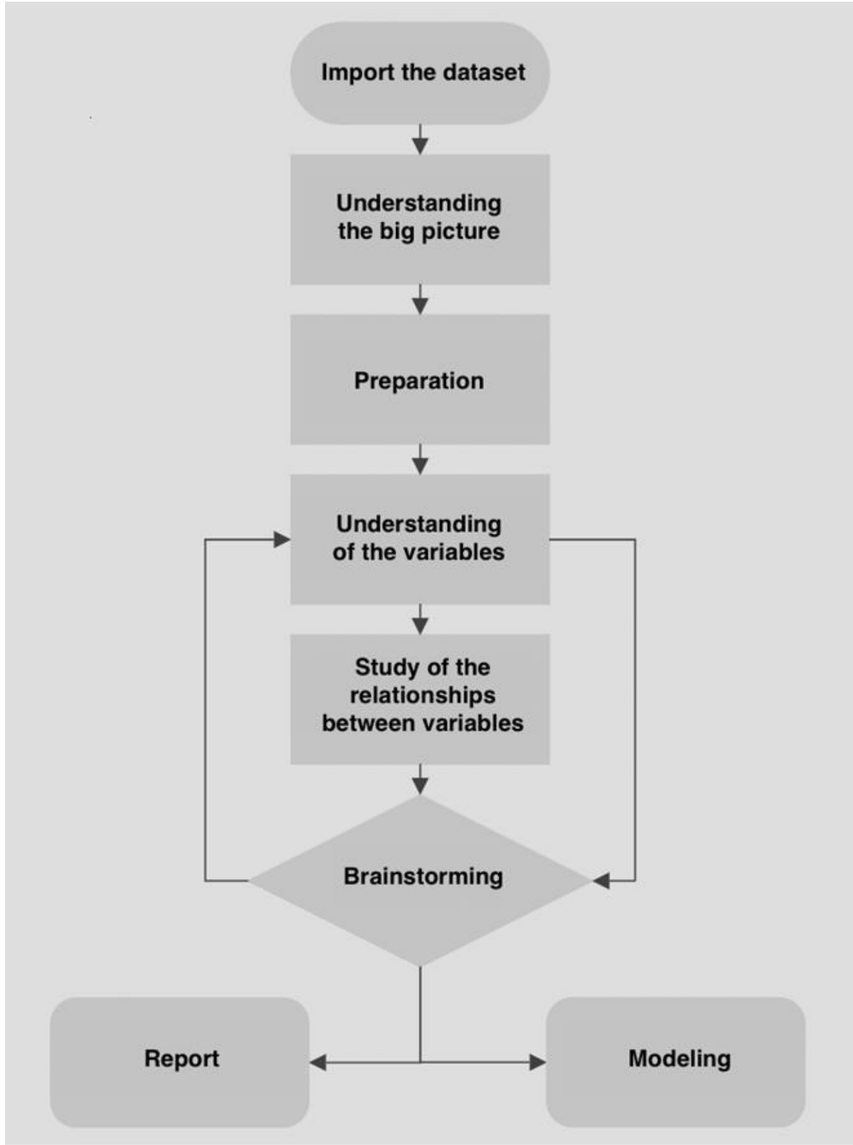
Data Understanding

1. Descarregar o dataset *wine.csv* disponível na Blackboard
2. Criar um novo notebook
3. Importar as bibliotecas

```
import sklearn as skl
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import preprocessing
```

1. Carregar o dataset para um dataframe usando a função read_csv()

```
'''
Load CSV
'''
df = pd.read_csv('wine.csv')
```



Data Understanding

5. Inspeccione os dados

```
'''
Inspect data
'''
print(df.columns)
```

```
Index(['Alcohol', 'Malic acid', 'Ash', 'Alcalinity of ash', 'Magnesium',
       'Total phenols', 'Flavanoids', 'Nonflavanoid phenols',
       'Proanthocyanins', 'Color intensity', 'Hue',
       'OD280/OD315 of diluted wines', 'Proline', 'Class'],
      dtype='object')
```

```
df.head()
```

	Alcohol	Malic acid	Ash	Alcalinity of ash	Magnesium	Total phenols	Flavanoids	Nonflavanoid phenols	Proanthocyanins	Color intensity	Hue	OD280/OD315 of diluted wines	Proline	Class
0	14.23	1.71	2.43	15.6	127	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065	one
1	13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050	one
2	13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185	one
3	14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18	7.80	0.86	3.45	1480	one
4	13.24	2.59	2.87	21.0	118	2.80	2.69	0.39	1.82	4.32	1.04	2.93	735	one

Data Understanding

5. Inspeccione os dados

```
df.tail()
```

	Alcohol	Malic acid	Ash	Alcalinity of ash	Magnesium	Total phenols	Flavanoids	Nonflavanoid phenols	Proanthocyanins	Color intensity	Hue	OD280/OD315 of diluted wines	Proline	Class
173	13.71	5.65	2.45	20.5	95	1.68	0.61	0.52	1.06	7.7	0.64	1.74	740	three
174	13.40	3.91	2.48	23.0	102	1.80	0.75	0.43	1.41	7.3	0.70	1.56	750	three
175	13.27	4.28	2.26	20.0	120	1.59	0.69	0.43	1.35	10.2	0.59	1.56	835	three
176	13.17	2.59	2.37	20.0	120	1.65	0.68	0.53	1.46	9.3	0.60	1.62	840	three
177	14.13	4.10	2.74	24.5	96	2.05	0.76	0.56	1.35	9.2	0.61	1.60	560	three

```
df.shape
```

```
(178, 14)
```



- O conjunto de dados possui 178 registos/linhas;
- Cada linha é caracterizada por 14 atributos/colunas;
- A classe tem 3 classificações possíveis: “one”, “two” e “three” que se referem ao tipo de vinho.

Data Understanding

6. Verifique se há valores nulos e o tipo de dados de cada atributo

```
#df.dtypes
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 178 entries, 0 to 177
Data columns (total 14 columns):
 #   Column                        Non-Null Count  Dtype  
---  ------                        --------------  -----  
 0   Alcohol                       178 non-null    float64
 1   Malic acid                    178 non-null    float64
 2   Ash                           178 non-null    float64
 3   Alcalinity of ash             178 non-null    float64
 4   Magnesium                     178 non-null    int64  
 5   Total phenols                 178 non-null    float64
 6   Flavanoids                    178 non-null    float64
 7   Nonflavanoid phenols          178 non-null    float64
 8   Proanthocyanins               178 non-null    float64
 9   Color intensity               178 non-null    float64
 10  Hue                           178 non-null    float64
 11  OD280/OD315 of diluted wines  178 non-null    float64
 12  Proline                       178 non-null    int64  
 13  Class                         178 non-null    object 
dtypes: float64(11), int64(2), object(1)
memory usage: 19.6+ KB
```

- Não existem valores nulos;
- Os atributos são todos numéricos, exceto a Classe.

Data Understanding

7. Verifique se há valores em falta

```
'''
Missing data
'''
df.isna().any()
```

```
Alcohol                         False
Malic acid                      False
Ash                             False
Alcalinity of ash               False
Magnesium                       False
Total phenols                   False
Flavanoids                      False
Nonflavanoid phenols            False
Proanthocyanins                 False
Color intensity                 False
Hue                             False
OD280/OD315 of diluted wines    False
Proline                         False
Class                           False
dtype: bool
```

```
print(df.isna().sum())
```

```
Alcohol                         0
Malic acid                      0
Ash                             0
Alcalinity of ash               0
Magnesium                       0
Total phenols                   0
Flavanoids                      0
Nonflavanoid phenols            0
Proanthocyanins                 0
Color intensity                 0
Hue                             0
OD280/OD315 of diluted wines    0
Proline                         0
Class                           0
dtype: int64
```

- Não existem valores em falta.

Data Understanding

8. Apresente algumas estatísticas descritivas (para todos os dados numéricos ou apenas os desejados)

```
df.describe()
```

	Alcohol	Malic acid	Ash	Alcalinity of ash	Magnesium	Total phenols	Flavanoids	Nonflavanoid phenols	Proanthocyanins	Color intensity	Hue	OD280/OD315 of diluted wines	Proline
count	178.000000	178.000000	178.000000	178.000000	178.000000	178.000000	178.000000	178.000000	178.000000	178.000000	178.000000	178.000000	178.000000
mean	13.000618	2.336348	2.366517	19.494944	99.741573	2.295112	2.029270	0.361854	1.590899	5.058090	0.957449	2.611685	746.893258
std	0.811827	1.117146	0.274344	3.339564	14.282484	0.625851	0.998859	0.124453	0.572359	2.318286	0.228572	0.709990	314.907474
min	11.030000	0.740000	1.360000	10.600000	70.000000	0.980000	0.340000	0.130000	0.410000	1.280000	0.480000	1.270000	278.000000
25%	12.362500	1.602500	2.210000	17.200000	88.000000	1.742500	1.205000	0.270000	1.250000	3.220000	0.782500	1.937500	500.500000
50%	13.050000	1.865000	2.360000	19.500000	98.000000	2.355000	2.135000	0.340000	1.555000	4.690000	0.965000	2.780000	673.500000
75%	13.677500	3.082500	2.557500	21.500000	107.000000	2.800000	2.875000	0.437500	1.950000	6.200000	1.120000	3.170000	985.000000
max	14.830000	5.800000	3.230000	30.000000	162.000000	3.880000	5.080000	0.660000	3.580000	13.000000	1.710000	4.000000	1680.000000

```
df['Color intensity'].describe()
```

```
count    178.000000
mean       5.058090
std        2.318286
min        1.280000
25%        3.220000
50%        4.690000
75%        6.200000
max       13.000000
Name: Color intensity, dtype: float64
```

Data Understanding

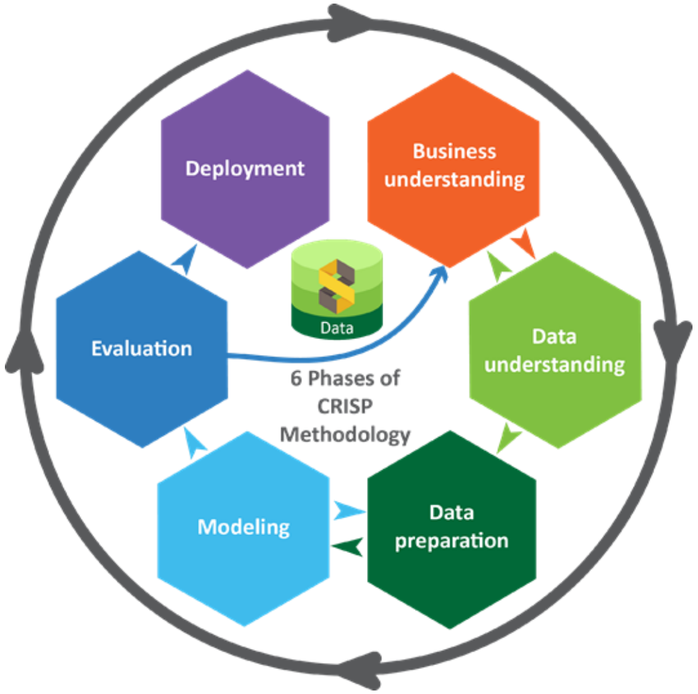
Com esta análise, conseguimos caracterizar melhor o conjunto de dados:

- Possui **178 entradas**;
- Contém **14 atributos** - 13 são propriedades físico-químicas do vinho e 1 é a classificação;
- Todos os valores são **não nulos**;
- **Não há valores em falta**.

O objetivo de trabalhar este conjunto de dados pode ser **prever** o **tipo de vinho** através das suas **propriedades/características** - o alvo é uma variável categórica numérica que abrange os valores um, dois e três.

Data Preparation

O que é o *Data Preparation* dentro de um processo de Data Science?



⚠ Os dados em bruto não podem ser utilizados diretamente pelos modelos!

Data Preparation transforma os dados num formato adequado para análise/modelagem. É a etapa mais **demorada** mas também a mais **importante** do processo.

Objetivo: tratar os dados no que diz respeito a:

- Selecionar os dados aplicáveis;
- Tratar os valores em falta, eliminando-os ou imputando-os;
- Construir novos dados, i.e., derivar novos elementos a partir dos existentes;
- Formatar os dados na estrutura pretendida;
- Eliminar colunas e características indesejadas;
- ...

Data Preparation

Consiste em várias etapas. Muitas vezes, vai ser necessário verificar a API da biblioteca que está a ser usada. Aqui estão alguns *links* para referência futura:

- Numpy (https://numpy.org/doc/stable/)
- Pandas (https://pandas.pydata.org/docs/)
- Matplotlib (https://matplotlib.org/stable/contents.html)
- Seaborn (https://seaborn.pydata.org/api.html)
- Scikit Learn (https://scikit-learn.org/stable/modules/classes.html)

Data Preparation

Pandas

E aqui estão algumas funções básicas do **Pandas** que precisaremos (em algum momento) no futuro para a preparação de dados:

pandas.DataFrame.drop

pandas.DataFrame.drop_duplicates

pandas.DataFrame.fillna

pandas.DataFrame.isna

pandas.DataFrame.interpolate

pandas.DataFrame.dropna

pandas.DataFrame.groupby

pandas.DataFrame.loc

pandas.DataFrame.iloc

...

Data Preparation

Scikit-Learn

E aqui estão algumas funções básicas do **Scikit-Learn** que precisaremos (em algum momento) no futuro para a preparação de dados:

sklearn.preprocessing.MinMaxScaler

sklearn.preprocessing.StandardScaler

sklearn.preprocessing.KBinsDiscretizer

sklearn.preprocessing.LabelEncoder

sklearn.feature_selection

sklearn.metrics

...

Data Preparation

1. Remover duplicados

```
'''
Drop Duplicates
'''
print(df.duplicated().sum())
print(df.drop_duplicates(inplace=True))
print(df.info())
```

```
0
None
<class 'pandas.core.frame.DataFrame'>
Int64Index: 178 entries, 0 to 177
Data columns (total 14 columns):
 #   Column                        Non-Null Count  Dtype
---  ------                        --------------  -----
 0   Alcohol                       178 non-null    float64
 1   Malic acid                    178 non-null    float64
 2   Ash                           178 non-null    float64
 3   Alcalinity of ash             178 non-null    float64
 4   Magnesium                     178 non-null    int64
 5   Total phenols                 178 non-null    float64
 6   Flavanoids                    178 non-null    float64
 7   Nonflavanoid phenols          178 non-null    float64
 8   Proanthocyanins               178 non-null    float64
 9   Color intensity               178 non-null    float64
 10  Hue                           178 non-null    float64
 11  OD280/OD315 of diluted wines  178 non-null    float64
 12  Proline                       178 non-null    int64
 13  Class                         178 non-null    object
dtypes: float64(11), int64(2), object(1)
memory usage: 20.9+ KB
None
```

Data Preparation

2. Renomear atributos

```
'''
Rename complicated columns' names
'''
df.rename(columns={"OD280/OD315 of diluted wines": "Protein Concentration"}, inplace=True)
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 178 entries, 0 to 177
Data columns (total 14 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   Alcohol                178 non-null    float64
 1   Malic acid             178 non-null    float64
 2   Ash                    178 non-null    float64
 3   Alcalinity of ash      178 non-null    float64
 4   Magnesium              178 non-null    int64
 5   Total phenols          178 non-null    float64
 6   Flavanoids             178 non-null    float64
 7   Nonflavanoid phenols   178 non-null    float64
 8   Proanthocyanins        178 non-null    float64
 9   Color intensity        178 non-null    float64
 10  Hue                    178 non-null    float64
 11  Protein Concentration  178 non-null    float64
 12  Proline                178 non-null    int64
 13  Class                  178 non-null    object
dtypes: float64(11), int64(2), object(1)
memory usage: 20.9+ KB
```

Data Preparation

3. Remover valores indesejados

```
'''
Remove values (Ash smaller than 2, Alcalinity bigger than 15)
'''
df_clean = df.drop(df.loc[(df['Ash']<2) & (df['Alcalinity of ash']>15)].index)
print(df_clean)
#df.drop(df.loc[(df['Ash']<2) & (df['Alcalinity of ash']>15)].index, inplace=True) ##alternative
```

```
     Alcohol  Malic acid   Ash  Alcalinity of ash  Magnesium  Total phenols  \
0      14.23        1.71  2.43               15.6        127           2.80
1      13.20        1.78  2.14               11.2        100           2.65
2      13.16        2.36  2.67               18.6        101           2.80
3      14.37        1.95  2.50               16.8        113           3.85
4      13.24        2.59  2.87               21.0        118           2.80
..       ...         ...   ...                ...        ...            ...
173    13.71        5.65  2.45               20.5         95           1.68
174    13.40        3.91  2.48               23.0        102           1.80
175    13.27        4.28  2.26               20.0        120           1.59
176    13.17        2.59  2.37               20.0        120           1.65
177    14.13        4.10  2.74               24.5         96           2.05

     Flavanoids  Nonflavanoid phenols  Proanthocyanins  Color intensity   Hue  \
0          3.06                  0.28             2.29             5.64  1.04
1          2.76                  0.26             1.28             4.38  1.05
2          3.24                  0.30             2.81             5.68  1.03
3          3.49                  0.24             2.18             7.80  0.86
4          2.69                  0.39             1.82             4.32  1.04
..          ...                   ...              ...              ...   ...
173        0.61                  0.52             1.06             7.70  0.64
174        0.75                  0.43             1.41             7.30  0.70
175        0.69                  0.43             1.35            10.20  0.59
176        0.68                  0.53             1.46             9.30  0.60
```



Uma vez que todas as características parecem ser medidas físico-químicas, todas elas podem ser úteis e ajudar a definir a segmentação do tipo de vinho. Não há motivo para remover colunas.

Exploratory Data Analysis (EDA)

- É uma etapa importante na ciência de dados, uma vez que **visualiza** os dados para compreender as suas principais **características**, encontrar **padrões** e descobrir como diferentes partes dos dados estão **relacionadas**.
- Dependendo do número de colunas que estamos a analisar, podemos dividir a EDA em dois tipos:

Univariate Analysis

- Análise de **uma única variável** de cada vez;
- Descrição e resumo dos dados;
- Não lida com causas e relações;
- Principais técnicas:
  - Medidas de tendência central (média, mediana, moda)
  - Medidas de dispersão (variância, desvio-padrão, amplitude)
  - Distribuições de frequência e histogramas

Exemplo: Idade média dos alunos de uma turma

Multivariate Analysis

- Estudo de **duas ou mais variáveis** ao mesmo tempo;
- Procura entender relações, padrões e dependências;
- Principais técnicas:
  - Regressão múltipla
  - Análise de correlação
  - PCA (Análise de Componentes Principais)
  - Clustering

Exemplo: Relação entre idade, horas de estudo e nota final

Univariate Analysis

1. Itere por cada *feature* relevante e obtenha informações básicas.

```
'''
Categorical variables
'''
df['Class'].value_counts(normalize=True)
```

```
two      0.398876
one      0.331461
three    0.269663
Name: Class, dtype: float64
```

```
'''
Numeric variables
'''
df['Magnesium'].describe()
```

```
count    178.000000
mean      99.741573
std       14.282484
min       70.000000
25%       88.000000
50%       98.000000
75%      107.000000
max      162.000000
Name: Magnesium, dtype: float64
```

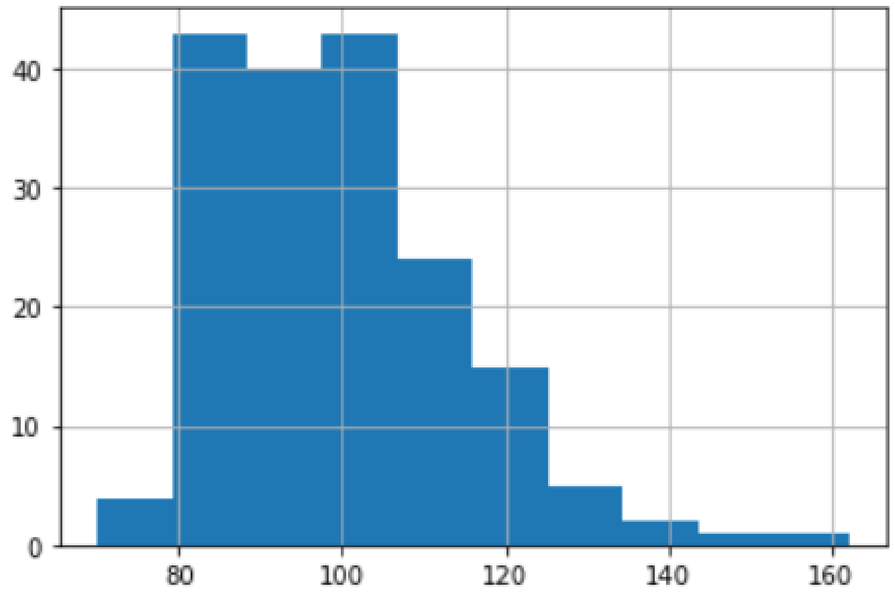
Univariate Analysis

Histograma

- Mostra a distribuição de dados numéricos
- Semelhante ao gráfico de barras, mas usado para dados contínuos
- Agrupa os valores em intervalos (bins)

```
print(f"Histogram: {df['Magnesium'].hist()}")
```

```
Histogram: AxesSubplot(0.125,0.125;0.775x0.755)
```



Não segue uma curva normal e apresenta picos.

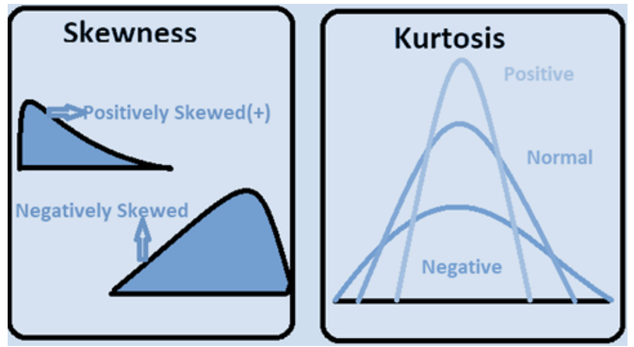
```
print(f"Skewness: {df['Magnesium'].skew()}")
```

```
Skewness: 1.098191054755161
```

```
print(f"Kurtosis: {df['Magnesium'].kurt()}")
```

```
Kurtosis: 2.1049913235905557
```

Os valores de Kurtosis e assimetria são superiores a 1.



Univariate Analysis

Resumo

Para variáveis categóricas:
Crie **tabelas de frequências**, **gráficos de barras** e **pie charts** para compreender a distribuição das categorias e identificar desequilíbrios ou padrões invulgares.

Para variáveis numéricas:
Crie **histogramas** e **boxplots** para visualizar a distribuição, forma, dispersão e potenciais valores discrepantes.

Agora é possível resumir o conjunto de dados criando um pequeno documento com informações detalhadas:

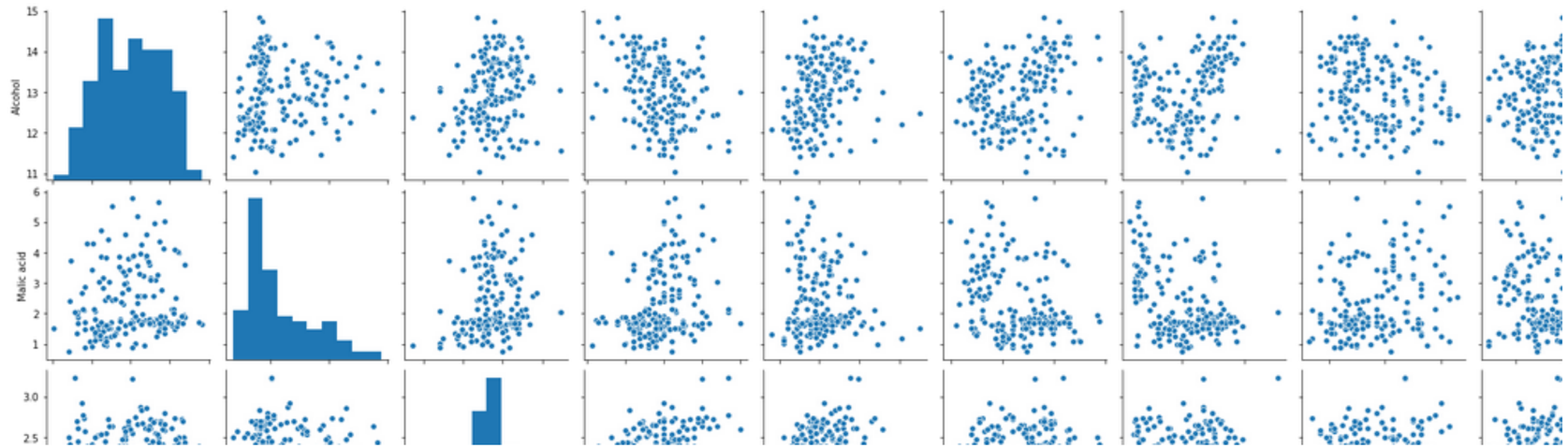
- ***Feature***: nome da característica;
- **Tipo**: o tipo ou formato da característica.
- **Contexto**: informações úteis para compreender o espaço semântico da característica. No caso deste conjunto de dados, o contexto é sempre físico-químico;
- **Expectativa**: qual é a relevância desta característica em relação à tarefa? Podemos usar uma escala «Alta, Média, Baixa»;
- **Comentários**: se temos ou não comentários a fazer sobre a característica.

Multivariate Analysis

1. Comece por ver a relação entre todas as características:

```
'''
All variables
'''
sns.pairplot(df)
```

```
<seaborn.axisgrid.PairGrid at 0x7f08ccac79d0>
```



Multivariate Analysis

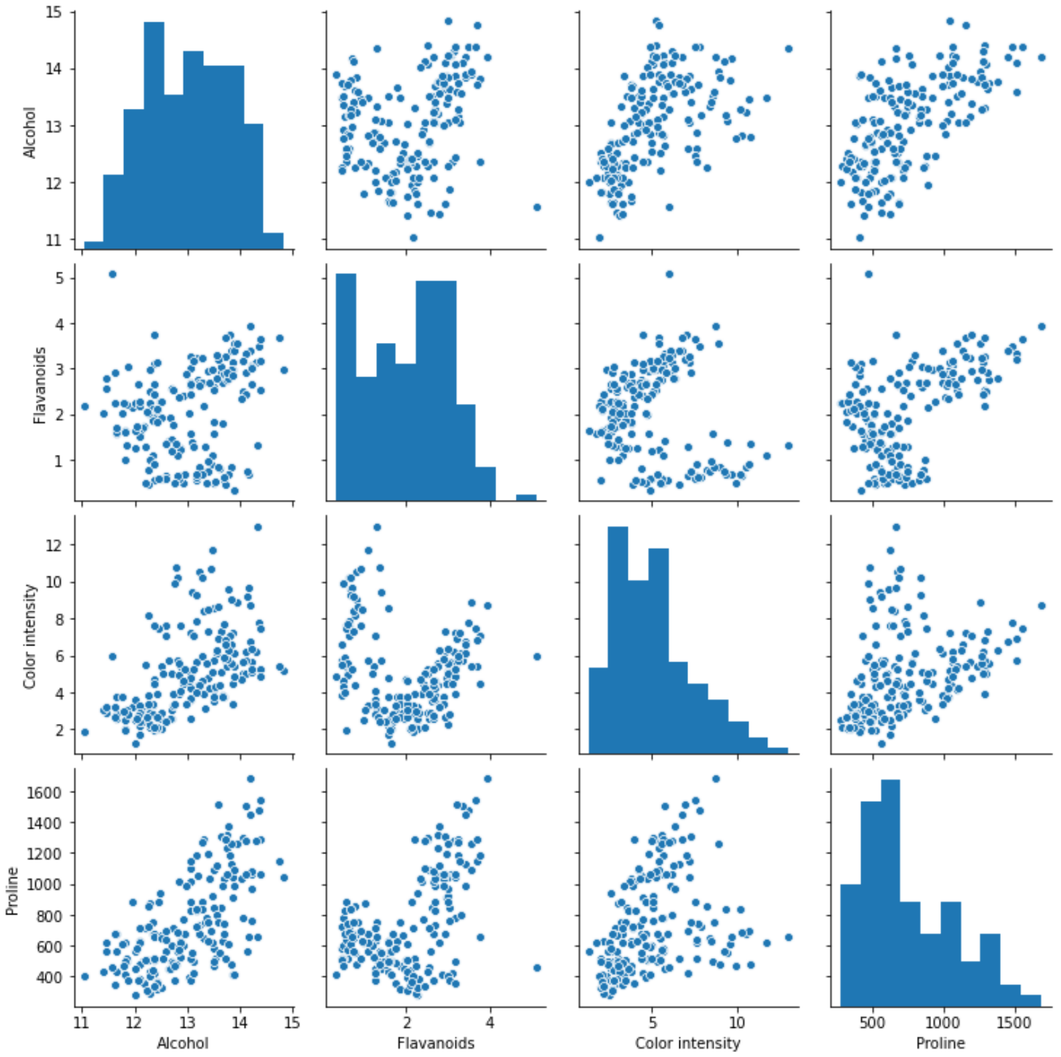
Pairplots

O que faz? Representa distribuições **univariadas** (na diagonal) e dispersões **entre pares** de variáveis

- Cada linha/coluna representa uma variável
- A **diagonal** mostra **histogramas**/densidades univariadas
- As outras células mostram **scatter plots** entre pares de variáveis
- Facilita a identificação de correlações, *clusters* e *outliers*
- Combina univariate e multivariate analysis em um só gráfico

Limitações:
- Difícil de ler com muitas variáveis (excesso de plots)
- Pode ficar pesado computacionalmente

```
'''
Relations
'''
cols = ['Alcohol', 'Flavanoids', 'Color intensity', 'Proline']
_ = sns.pairplot(df[cols], height = 2.5)
```



Multivariate Analysis

2. É possível agrupar por *feature*. Vamos utilizar a *Class*:

```
'''
Grouping
'''
df.groupby(by=['Class']).mean()
```

Class	Alcohol	Malic acid	Ash	Alcalinity of ash	Magnesium	Total phenols	Flavanoids	Nonflavanoid phenols	P
one	13.744746	2.010678	2.455593	17.037288	106.338983	2.840169	2.982373	0.290000	
three	13.153750	3.333750	2.437083	21.416667	99.312500	1.678750	0.781458	0.447500	
two	12.278732	1.932676	2.244789	20.238028	94.549296	2.258873	2.080845	0.363662	

Multivariate Analysis

3. Ou *features*. Considere agora a *Class* e a *Proline*:

```
df.groupby(by=['Class', 'Proline']).mean()
```

Class	Proline	Alcohol	Malic acid	Ash	Alcalinity of ash	Magnesium	Total phenols	Flavanoids	Nonflavanoid phenols
one	680	13.240	3.980	2.290	17.5	103.0	2.640	2.630	0.32
	735	13.240	2.590	2.870	21.0	118.0	2.800	2.690	0.39
	760	14.220	3.990	2.510	13.2	128.0	3.000	3.040	0.20
	770	12.930	3.800	2.650	18.6	102.0	2.410	2.410	0.25
	780	14.060	1.630	2.280	16.0	126.0	3.000	3.170	0.24
...	...	...	...	...	...	...	...	...	...
two	750	12.835	0.965	2.155	15.9	123.0	2.215	1.575	0.45

Multivariate Analysis

4. Com múltiplas agregações:

```
print(df.groupby(by=['Alcohol']).agg(pd.Series.mode))
```

```
          Malic acid          Ash Alcalinity of ash Total phenols  \
Alcohol
11.03           1.51          2.2              21.5          2.46
11.41           0.74          2.5                21          2.48
11.45            2.4         2.42                20           2.9
11.46           3.74         1.82              19.5          3.18
11.56           2.05         3.23              28.5          3.18
...              ...          ...               ...           ...
14.37           1.95          2.5              16.8          3.85
14.38    [1.87, 3.59]  [2.28, 2.38]     [12.0, 16.0]   [3.25, 3.3]
14.39           1.87         2.45              14.6           2.5
14.75           1.73         2.39              11.4           3.1
14.83           1.64         2.17                14           2.8

          Flavanoids Nonflavanoid phenols Proanthocyanins Color intensity  \
Alcohol
11.03           2.17                0.52            2.01             1.9
11.41           2.01                0.42            1.44            3.08
11.45           2.79                0.32            1.83            3.25
```

```
print(df.groupby(by=['Alcohol', 'Flavanoids']).mean(numeric_only=True))
```

```
                    Malic acid   Ash  Alcalinity of ash  Magnesium  \
Alcohol Flavanoids
11.03   2.17              1.51  2.20               21.5         85
11.41   2.01              0.74  2.50               21.0         88
11.45   2.79              2.40  2.42               20.0         96
11.46   2.58              3.74  1.82               19.5        107
11.56   5.08              2.05  3.23               28.5        119
...                        ...   ...                ...        ...
14.38   3.17              3.59  2.28               16.0        102
        3.64              1.87  2.38               12.0        102
14.39   2.52              1.87  2.45               14.6         96
14.75   3.69              1.73  2.39               11.4         91
14.83   2.98              1.64  2.17               14.0         97

                    Total phenols  Nonflavanoid phenols  Proanthocyanins  \
Alcohol Flavanoids
11.03   2.17                 2.46                  0.52             2.01
11.41   2.01                 2.48                  0.42             1.44
11.45   2.79                 2.90                  0.32             1.83
11.46   2.58                 3.18                  0.24             3.58
11.56   5.08                 3.18                  0.47             1.87
...                           ...                   ...              ...
14.38   3.17                 3.25                  0.27             2.19
        3.64                 3.30                  0.29             2.96
```

Multivariate Analysis

Data Binning

O que faz? Agrupa **valores contínuos** em **categorias/intervalos/bins**

- Reduz ruído em variáveis contínuas
- Facilitar a interpretação de padrões
- Suavização de dados
- Mitigação de *outliers*

Limitações:
- Perda de informação
- Subjetividade
- Risco de sobreajuste

```
'''
Bins
'''
# https://scikit-learn.org/stable/modules/preprocessing.html#discretization
estimator = preprocessing.KBinsDiscretizer(n_bins=3, encode='ordinal', strategy='quantile')
df['alcohol_binned'] = estimator.fit_transform(df[['Alcohol']])
print('Bin Edges')
print(estimator.bin_edges_[0])
print('Alcohol Groups')
print(df.groupby(by=['alcohol_binned']).count())
```

```
Bin Edges
[11.03 12.52 13.48 14.83]
Alcohol Groups
                Alcohol  Malic acid  Ash  Alcalinity of ash  Magnesium  \
alcohol_binned
0.0                  59          59   59                 59         59
1.0                  58          58   58                 58         58
2.0                  61          61   61                 61         61

                Total phenols  Flavanoids  Nonflavanoid phenols  \
alcohol_binned
0.0                        59          59                    59
1.0                        58          58                    58
2.0                        61          61                    61

                Proanthocyanins  Color intensity  Hue  Protein Concentration  \
alcohol_binned
0.0                          59               59   59                     59
1.0                          58               58   58                     58
```

Multivariate Analysis

Data Binning

Tipos de binning

Equal-width

intervalos do mesmo tamanho, determinados por:

$$Largura\ do\ Intervalo = \frac{Valor\ \max - Valor\ Min}{n}$$

Vantagens: Simples de implementar e fácil de compreender.

Desvantagens: Pode resultar em segmentos com distribuição de dados altamente desigual.

Equal-frequency

cada intervalo contém um mesmo número aproximado de observações

Vantagens: garante tamanhos de intervalo equilibrados, evitando intervalos esparsos.

Desvantagens: a largura do intervalo pode variar significativamente.

Custom bins

definidos com base em conhecimento do domínio

Variável: Idade (0 a 100 anos)
Bins:

- 0–17 → Criança/Adolescente
- 18–39 → Jovem adulto
- 40–64 → Adulto
- 65+ → Idoso

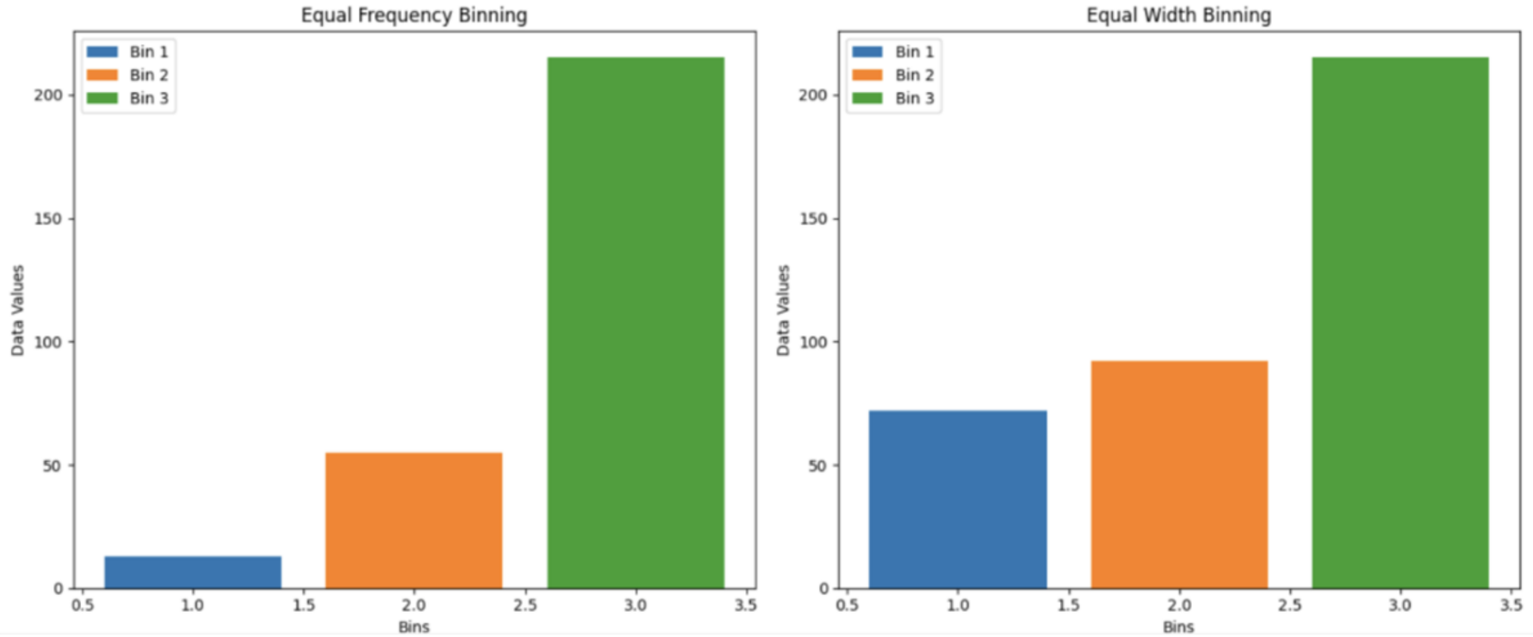
Multivariate Analysis

Data Binning

Equal Frequency Binning vs. Equal Width Binning

```
Equal Frequency Binning: [[5, 10, 11, 13], [15, 35, 50, 55], [72, 92, 204, 215]]

Equal Width Binning: [[5, 10, 11, 13, 15, 35, 50, 55, 72], [92], [204, 215]]
```



Multivariate Analysis

➡ Dispersão Estatística

O que faz? Mostra como os dados se distribuem em torno da **média**

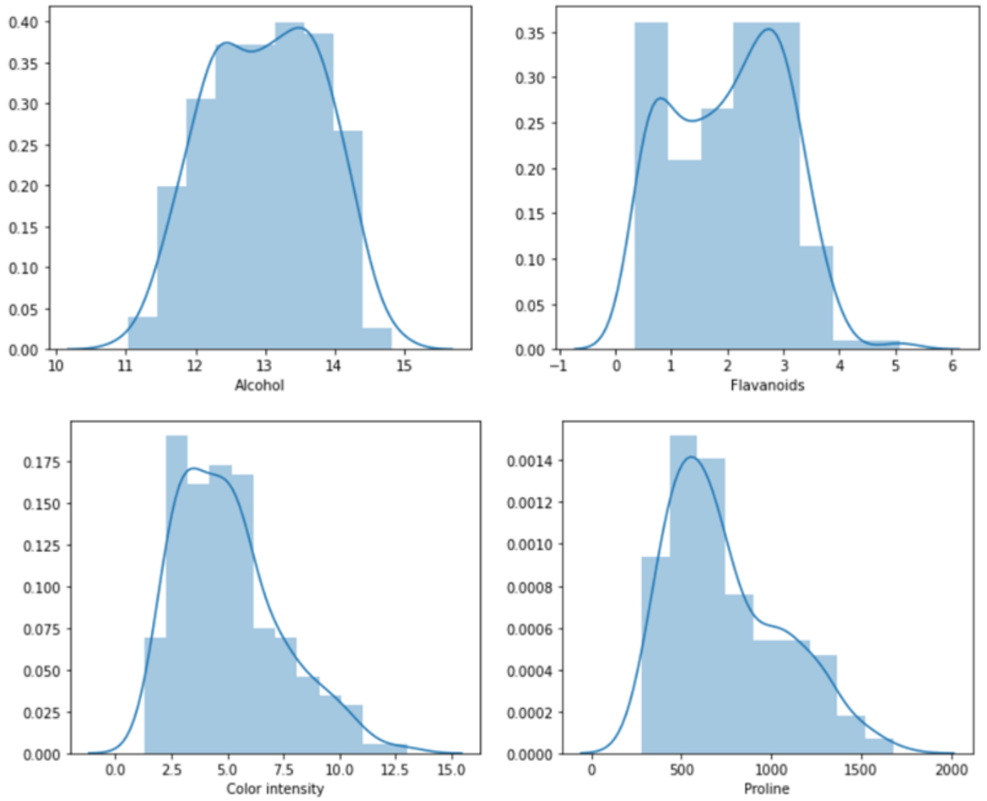
- Cria gráficos de distribuição (**histograma**, **KDE** ou ambos)
- Permite visualizar a **forma da distribuição** e o **grau de dispersão**
- Identificar consistência ou heterogeneidade num conjunto de dados
- Ajuda a interpretar médias (mostra se os dados estão **concentrados** ou **espalhados**)
- Deteta presença de *outliers* e irregularidades
- Fácil comparação entre grupos/categorias

Limitações:

- Algumas medidas são sensíveis a valores extremos (ex.: amplitude, variância).

```
'''
Statistical Dispersion
'''
fig, axs = plt.subplots(2, 2, figsize=(12, 10))
fig.suptitle('Histograms')

sns.histplot(df['Alcohol'], ax=axs[0, 0], kde=True, stat='density')
sns.histplot(df['Flavanoids'], ax=axs[0, 1], kde=True, stat='density')
sns.histplot(df['Color intensity'], ax=axs[1, 0], kde=True, stat='density'
sns.histplot(df['Proline'], ax=axs[1, 1], kde=True, stat='density')
```



Multivariate Analysis

Boxplots

O que faz? Representa a distribuição de uma variável através dos seus **quartis, mediana e valores máximo e mínimo**.

- Mostrar tendência central, dispersão e *outliers* de forma clara;
- Facilitar a compreensão da relação entre características numéricas e categóricas;
- Identificação rápida de assimetrias e *outliers;*

Limitações:
- Não mostra detalhes da distribuição (picos, caudas, multimodalidade);
- Pode ser mal interpretado sem contexto estatístico.

Um gráfico de caixa fornece um resumo de **cinco** números:

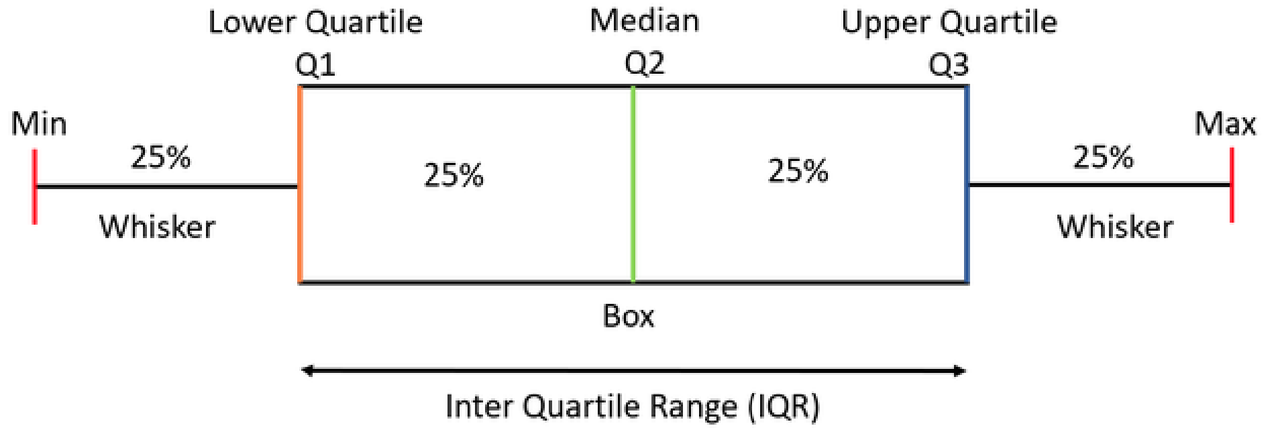
Mínimo - É o menor valor, excluindo os valores atípicos.

Primeiro quartil (Q1) - 25% dos dados abaixo do primeiro quartil

Mediana (Q2) - É o ponto médio do conjunto de dados. Metade dos valores está abaixo dele e metade acima.

Terceiro quartil (Q3) - 75% dos dados abaixo do terceiro quartil

Máximo - É o maior valor, excluindo os valores atípicos.

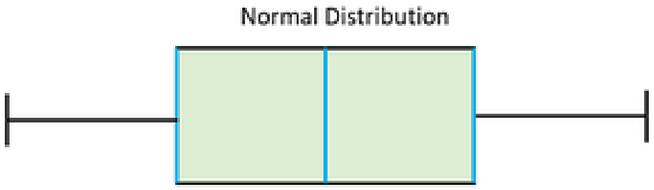


⚠ Os gráficos podem apresentar assimetria e a mediana pode não estar no centro da caixa.

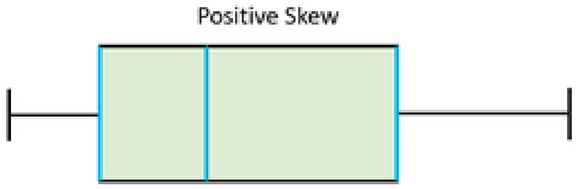
Multivariate Analysis

Boxplots

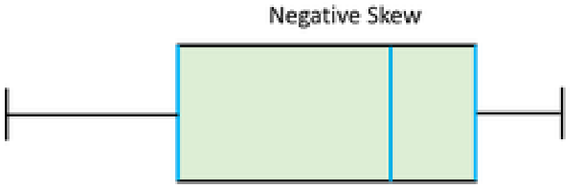
⚠ Os gráficos podem apresentar assimetria e a mediana pode não estar no centro da caixa.



Se a mediana estiver no centro da caixa e as arestas forem quase iguais em ambas as extremidades, os dados serão distribuídos normalmente.



Se a mediana estiver mais próxima do Q1 e se a fronteira na extremidade inferior for mais pequena. então terá uma assimetria positiva (assimetria à direita).



Se a mediana estiver mais próxima do Q3 e se o bordo na extremidade superior for menor, então terá uma assimetria negativa (assimetria à esquerda).

- A área dentro da caixa (50% dos dados) é conhecida como **Intervalo Interquartil (IQR)**

IQR= Q3-Q1

- Os ***outlies*** são os pontos de dados abaixo e acima dos limites inferior e superior

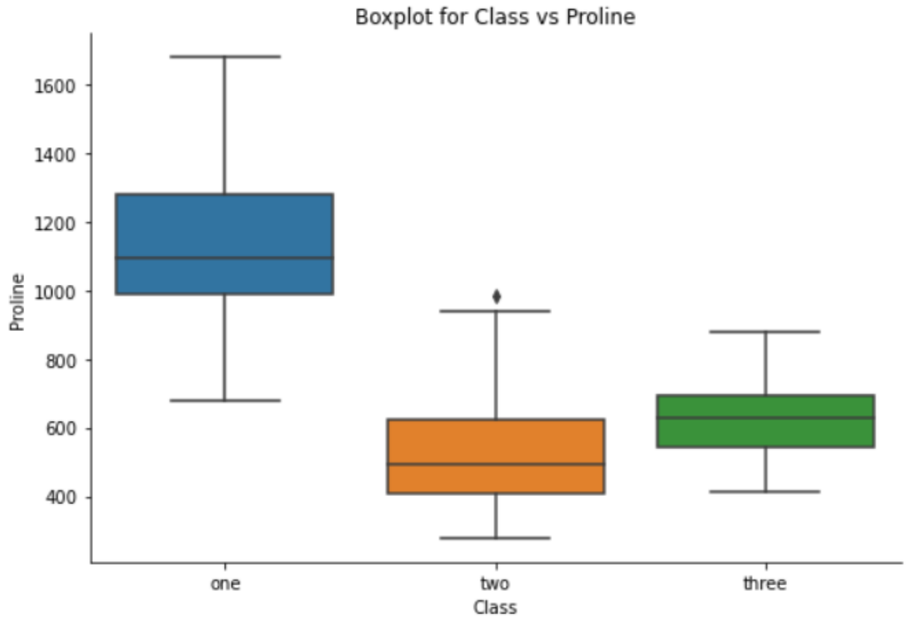
Limite Inferior = Q1 – 1,5*IQR

Limite Superior = Q3 + 1,5*IQR

Multivariate Analysis

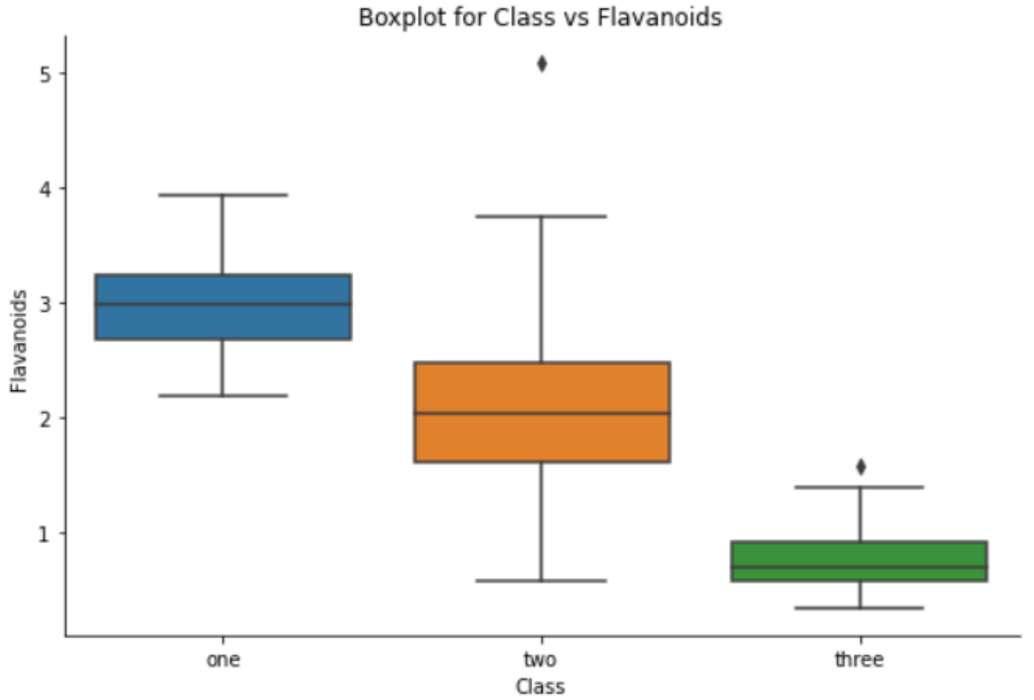
Boxplots

```
'''
Box plots (Outliers)
'''
sns.catplot(x="Class", y="Proline", data=df, kind="box", aspect=1.5)
plt.title("Boxplot for Class vs Proline")
plt.show()
```



A variável **Proline é um forte discriminador entre as classes**. Classe 1 destaca-se nitidamente pelas concentrações mais elevadas.

```
sns.catplot(x="Class", y="Flavanoids", data=df, kind="box", aspect=1.5)
plt.title("Boxplot for Class vs Flavanoids")
plt.show()
```



A variável **Flavanoids também separa bem as classes**, especialmente Classe 1 (valores altos) e Classe 3 (valores baixos).

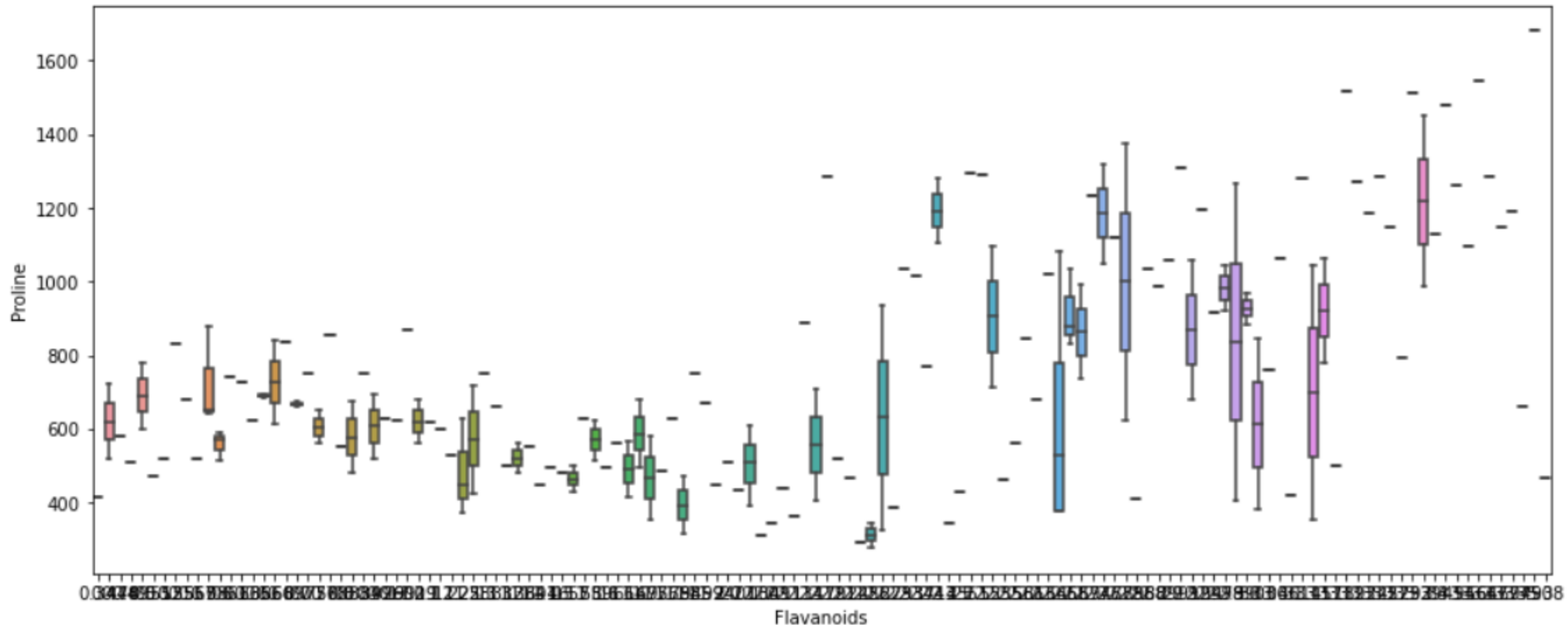
Multivariate Analysis

Boxplots

- Para cada valor (ou grupo de valores) de *Flavanoids*, o boxplot mostra a **distribuição de Proline.**

```
_, ax = plt.subplots(figsize=(15, 6))
fig.suptitle('Boxplot for Flavanoids vs Proline')
sns.boxplot(x=df["Flavanoids"], y=df["Proline"])
```

```
<AxesSubplot:xlabel='Flavanoids', ylabel='Proline'>
```



- A variabilidade de Proline aumenta com valores mais altos de Flavanoids;
- Existem muitos outliers ao longo do gráfico.

Multivariate Analysis

Scatterplots

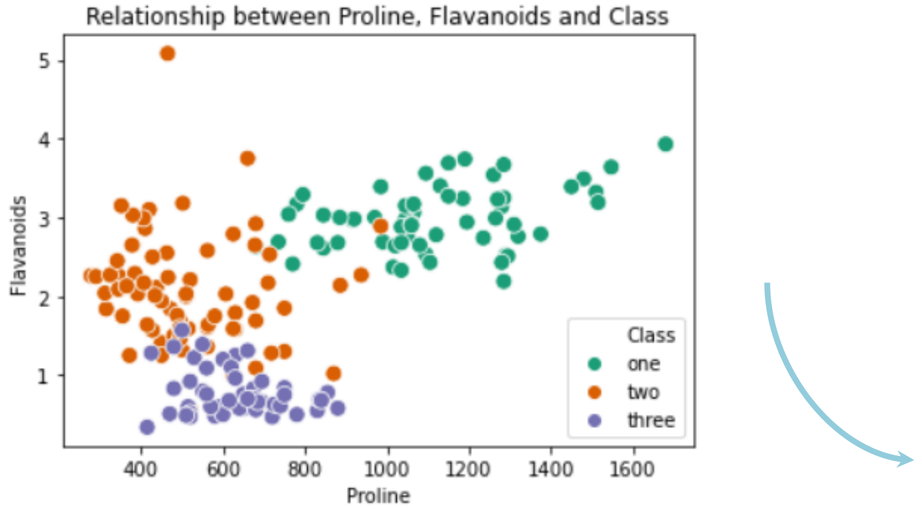
O que faz? Mostra a **relação** entre duas **variáveis quantitativas**.

- Cada ponto representa uma observação (x, y)
- Pode mostrar agrupamentos ou *outliers*
- Mostra padrões que não seriam claros em tabelas
- Útil para detetar correlações positivas/negativas
- Deteta presença de *outliers* e irregularidades.

Limitações:

- Difícil de interpretar com muitas variáveis ao mesmo tempo
- Não indica causalidade, apenas associação

```
'''
Scatter plots
'''
sns.scatterplot(x="Proline", y="Flavanoids", hue="Class", data=df, palette="Dark2", s=80)
plt.title("Relationship between Proline, Flavanoids and Class")
plt.show()
```

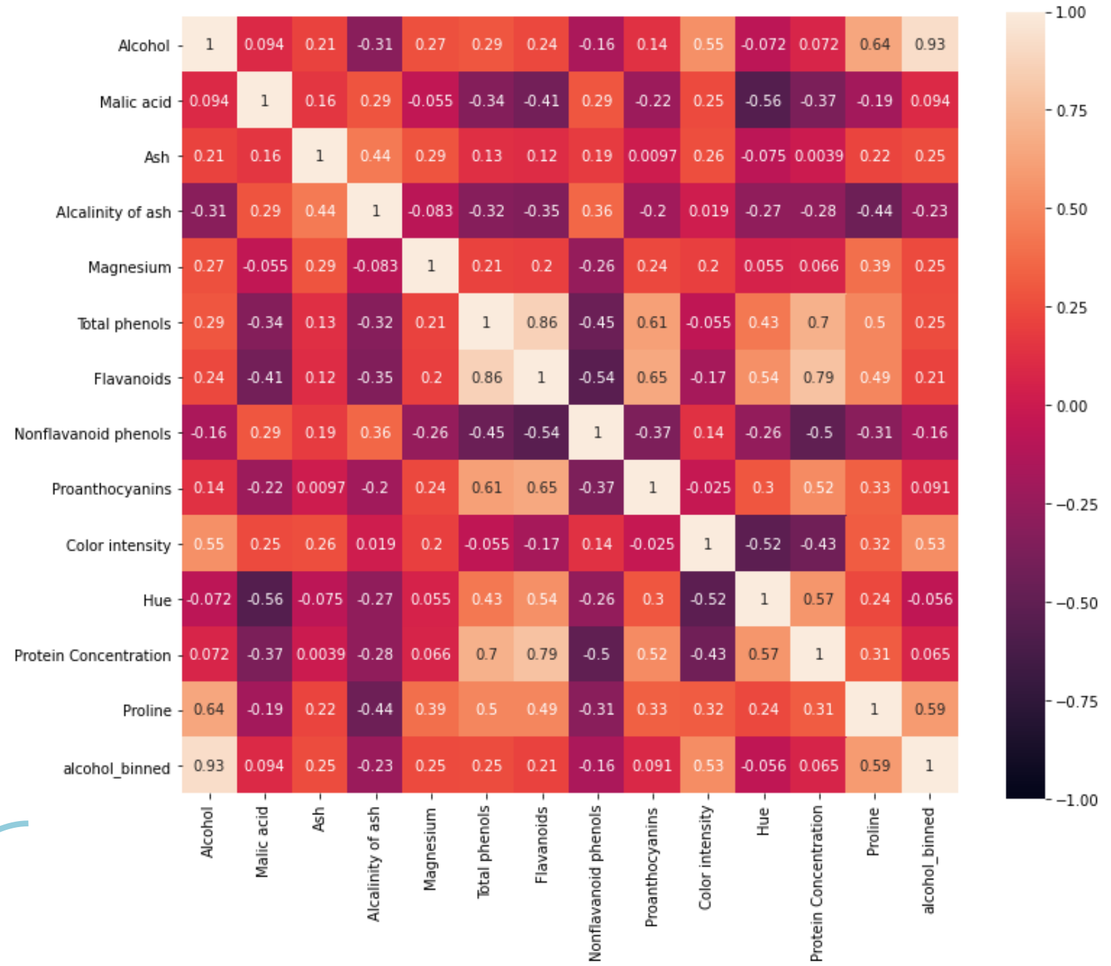


Para a classe um, os níveis de prolina são muito mais elevados, enquanto o nível de flavonóides permanece estável em torno do valor 3.

Multivariate Analysis

Heatmap

```
'''
Correlation
'''
corr_matrix = df.corr(numeric_only=True)
f, ax = plt.subplots(figsize=(12, 10))
sns.heatmap(corr_matrix, vmin=-1, vmax=1, square=True, annot=True);
```



O que faz? Representa graficamente a **correlação** entre variáveis

- Facilita a identificação de **relações fortes, fracas ou inexistentes**
- Mede a intensidade e direção da relação entre duas variáveis
- Varia de -1 a +1:
  - **+1** → correlação positiva perfeita
  - **0** → ausência de correlação
  - **-1** → correlação negativa perfeita

Limitações:

- Mostra apenas relações lineares
- Correlação ≠ causalidade (não implica causa e efeito)

Existe uma correlação muito forte entre o álcool e a prolina. Níveis elevados de álcool correspondem a níveis elevados de prolina.

Aplicação Prática

Ficha de exercícios 2



Blackboard